FOAF 人物词表 · 规范速查

FOAF — PEOPLE, AGENTS & PROFILE DOCUMENTS

01 总览 · 描述「人」的第一套词表

PART 1 · OVERVIEW

FOAF(Friend of a Friend)是描述人、组织及其社会关系的词表:名字邮箱头像、认识谁、属于哪个组、做了哪些东西。命名空间 `foaf:` = `http://xmlns.com/foaf/0.1/`。四层结构:

行动者	Agent · 人 / 组织 / 群组 / 账号	一切的根
身份标识	mbox · homepage · 相当于主键的 IFP	认人不认名
关系网络	knows 认识 · member 隶属 · made 作品	图就是这样长出来的
档案文档	PersonalProfileDocument · primaryTopic	「我」的出处

主线:认人不靠名字靠标识——mbox 这类「反向函数」属性两个人填了同一个值,推理机就判同一人。

本卡结构 = 本图:概览 → 类与身份 → 关系与文档 → 融合与实操。

02 定位 · 社区词表的活化石

PART 1 · POSITION

FOAF 从 2000 年做到今天,是语义网最早破圈的应用词表。它**不是 W3C 标准**——由 FOAF 项目(Dan Brickley、Libby Miller)以开源社区方式维护,当前版本 0.99「Paddington Edition」(2014-01-14),但命名空间固定在 `0.1/` 不随版本变。

制度	说明
术语状态制	每个词带状态:stable → 可放心用;testing → 可用会微调;archaic → 别再新增(如 firstName / lastName,换 givenName / familyName)
引用姿势	写 `foaf:` = xmlns.com/foaf/0.1/,别把 0.99 写进 URI;版本只在 spec 文档层面

03 类层级 · 一切皆行动者

PART 1 · CLASS TREE

树的根是 `foaf:Agent`——「会做事的东西」,人、组织、群组甚至软件都算。

类	一句话
foaf:Agent	一切行动者的根:人、组织、群组、软件
foaf:Person	人(在世与否、真实虚构皆可);Agent 的子类
foaf:Organization	公司、社团等建制化群体
foaf:Group	行动者的集合,自身也是 Agent(可用 foaf:member 列成员)
foaf:Document	广义文档;与 Organization、Project 不相交
foaf:Image	Document 的子类——图片也是文档
foaf:OnlineAccount	网络服务账号(子类:聊天/购物/游戏账号)
foaf:PersonalProfileDocument	「个人档案文档」:maker 和 primaryTopic 都是同一个人
foaf:Project	集体事业;与 Document、Person 不相交

04 身份标识 · 带「主键」性质的属性

PART 2 · IDENTITY PROPERTIES

名字可以重复,标识不行。FOAF 给一批属性标了**反向函数(IFP)**:两个 Agent 在该属性上取值相同,推理机即判为同一实体。

属性	用途	性质
`foaf:mbox`	个人邮箱(mailto: URI)	IFP · 认同一个人的主键
`foaf:mbox_sha1sum`	邮箱 URI 的 SHA1——隐私版 mbox,同样能认人	IFP
`foaf:homepage`	主页	IFP
`foaf:weblog` · `foaf:openid`	博客 / OpenID	IFP
`foaf:name` / `givenName` / `familyName` / `nick`	姓名拆分与昵称	无 IFP——名字不当主键
`foaf:gender` · `foaf:birthday` · `foaf:age`	人口学字段	gender 为函数属性

IFP 家族

除上表外,logo、jabberID、aimChatID 等 IM 账号也是 IFP。共同点:**值由本人独占**——所以同值即可同人。这条语义在合并数据源时威力巨大,见 §07。

05 社交与组织 · 图从哪长出来

PART 2 · SOCIAL & GROUPS

关系属性把孤立的人物节点连成图:

属性	连什么	备注
`foaf:knows`	Person → Person	domain/range 都是 Person;隐含「有一定双向互动」,方向从书写者视角定
`foaf:member`	Group → Agent	群组列成员;成员本身也可以是 Group
`foaf:made` / `foaf:maker`	Agent ↔ 造物	一对互逆:做了什么 / 谁做的
`foaf:depiction` / `foaf:img`	Thing → Image	img 是 depiction 的子属性(限 Person)
`foaf:account`	Agent → OnlineAccount	配 accountName / accountServiceHomepage 描述账号
`foaf:interest` · `foaf:currentProject`	Agent → 主题 / 项目	兴趣指向「关于该主题的 Document」
`foaf:workplaceHomepage`	Person → Document	雇主主页;另有 schoolHomepage

06 文档三件套 · 名字从哪来

PART 2 · DOCUMENT MACHINERY

「文档」与「主题」的关系是 FOAF 最精巧的部分:一条 subPropertyOf 链把主页、档案页、一般页面串起来。

`CHAIN · homepage ⊑ isPrimaryTopicOf ⊑ page`

`page`:这份文档**谈到**它;`isPrimaryTopicOf`:这份文档**主要谈**它;`homepage`:既是主页、又主要谈它、还由它独占——三重身份一步到位。

三对互逆 写一边得另一边

- primaryTopic ↔ isPrimaryTopicOf:文档的主主题 / 主题的主文档
- maker ↔ made:造物者 / 造物
- depiction ↔ depicts:被描绘 / 描绘

PersonalProfileDocument 自我声明的闭环

- 定义:maker 和 primaryTopic 是同一个人的 Document
- 「这份档案是我写的、写的是我」——档案可信度的形式化根基(WebID 档案即此形态)

07 IFP 的杀伤力 · 合并数据源

PART 3 · MERGING BY IFP

IFP 是 FOAF 在图谱融合里真正的杀手锏:两个数据源各自描述「张三」,只要 `mbox` 相同,OWL 推理直接产出 `owl:sameAs`,两张人物卡自动合并。

```
# 数据源 A(公司通讯录)          # 数据源 B(社区 wiki)
ex:a_103 foaf:mbox <mailto:zx@ex.com> .   wiki:p_7 foaf:mbox <mailto:zx@ex.com> .
ex:a_103 foaf:name "张三" .               wiki:p_7 foaf:nick "zx" .

# 推理机(mbox 为 IFP)⇒
ex:a_103 owl:sameAs wiki:p_7 .   # 两卡合一,名字与昵称落到同一实体
```

双刃剑

IFP 合并是**语义级自动合并**——错配一个旧邮箱就会把两个人焊死成一人,而且推理结论难以拆开。生产环境先对 IFP 值做质量校验,再开推理。

08 一份个人档案 · Turtle

PART 3 · A WORKED PROFILE

```
<http://ex.com/~zx/foaf.ttl> a foaf:PersonalProfileDocument ;
    foaf:maker ex:zx ;                # 我写的
    foaf:primaryTopic ex:zx .           # 写的是我 —— 闭环成立

ex:zx a foaf:Person ;
    foaf:name "张三" ;
    foaf:givenName "三" ; foaf:familyName "张" ;
    foaf:mbox <mailto:zx@ex.com> ;        # 主键,慎改
    foaf:homepage <http://ex.com/~zx/> ;
    foaf:workplaceHomepage <http://ex.com/> ;
    foaf:knows ex:李四 , dbpedia:Wang_Anshi ;
    foaf:account ex:zx_github ;
    foaf:interest <https://www.w3.org/2001/sw/> .

ex:zx_github a foaf:OnlineAccount ;
    foaf:accountName "zx" .
```

09 实操坑清单

PART 4 · PITFALLS

01 **mbox 一旦发布别换:**它是 IFP 主键——换邮箱等于换身份,历史数据的 sameAs 全部失效;邮箱不想公开就用 mbox_sha1sum。

02 **knows 不对称:**我写 knows 你,推不出你知道我;要双向就写两条。

03 **archaic 词别新增:**firstName / lastName / surname 已弃,统一 givenName / familyName。

04 **img ⊏ depiction:**给人物配头像写 img(更具体),给一般事物写 depiction。

05 **Disjoint 别踩:**Person∩Project、Document∩Organization 等互不相交——把公司同时写成 Organization 和 Project 会判不一致。

06 **主页链写对的层级:**个人主页写 homepage(自动得到 page 与 isPrimaryTopicOf),别倒过来只写 page。

07 **Group 自身是 Agent:**可以把 Group 当关系的主语(made / account 都合法);列出成员用 member。

08 **与 schema.org 对接:**schema.org:Person 与 foaf:Person 语义对应,跨词表映射用 owl:equivalentClass 声明,别两套属性混写同一份档案。